Выявление и устранение медленных запросов с помощью SQL Server Profiler

Предметная область: сотрудники, отделы, проекты и табели. **СУБД:** Microsoft SQL Server 2025 (17.0.1000.7), SSMS 22, SQL Server Profiler 22. В задании указана версия 2019, на компьютере установлена 2025, порядок работы тот же. **Имя БД:** TrainingDB. **Скрипты:** папка `скрипты/18_10`.

1. Цель работы

Научиться запускать трассировку в SQL Server Profiler, отбирать события, фильтровать лишнее, находить медленные запросы и блокировки, анализировать планы, предлагать и проверять меры оптимизации.

2. Описание базы

Таблица	Строк	Страниц данных
Departments	5	2
Employees	500	7
Projects	3	2
Timesheets	2000	10

Схема и данные взяты из задания (`01_setup.sql`), значения сотрудников и табелей случайные. Даты записаны как `'20230101'`, потому что логин с русским языком может неверно разобрать формат с дефисами. Запрос A отбирает 256 из 500 сотрудников, запрос B 2000 из 2000 строк табеля, запрос C 5 строк по отделам.

3. Настройки трассировки

Имя: TrainingDB_Profiler_2019. **Шаблон:** Blank.

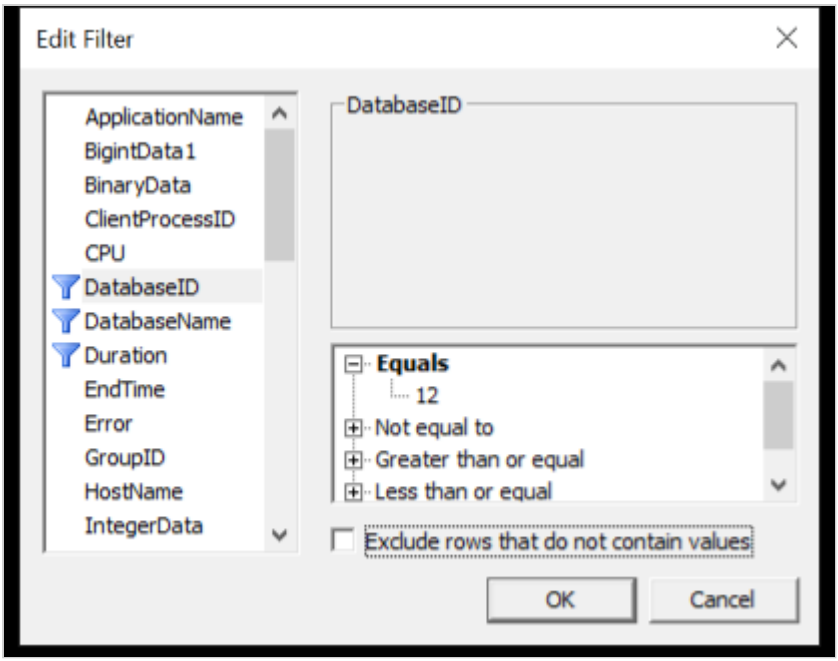
Событие	Колонки
RPC:Completed, SQL:BatchCompleted	TextData, Duration, Reads, CPU, RowCounts, DatabaseName, DatabaseID
Showplan Statistics Profile	BinaryData, DatabaseName, DatabaseID
Showplan XML Statistics Profile	TextData, DatabaseName, DatabaseID
Lock:Acquired	TextData, Duration, Mode, ObjectID, DatabaseID

Фильтры: DatabaseName Like TrainingDB, Duration >= 100 мс, DatabaseID = 12.

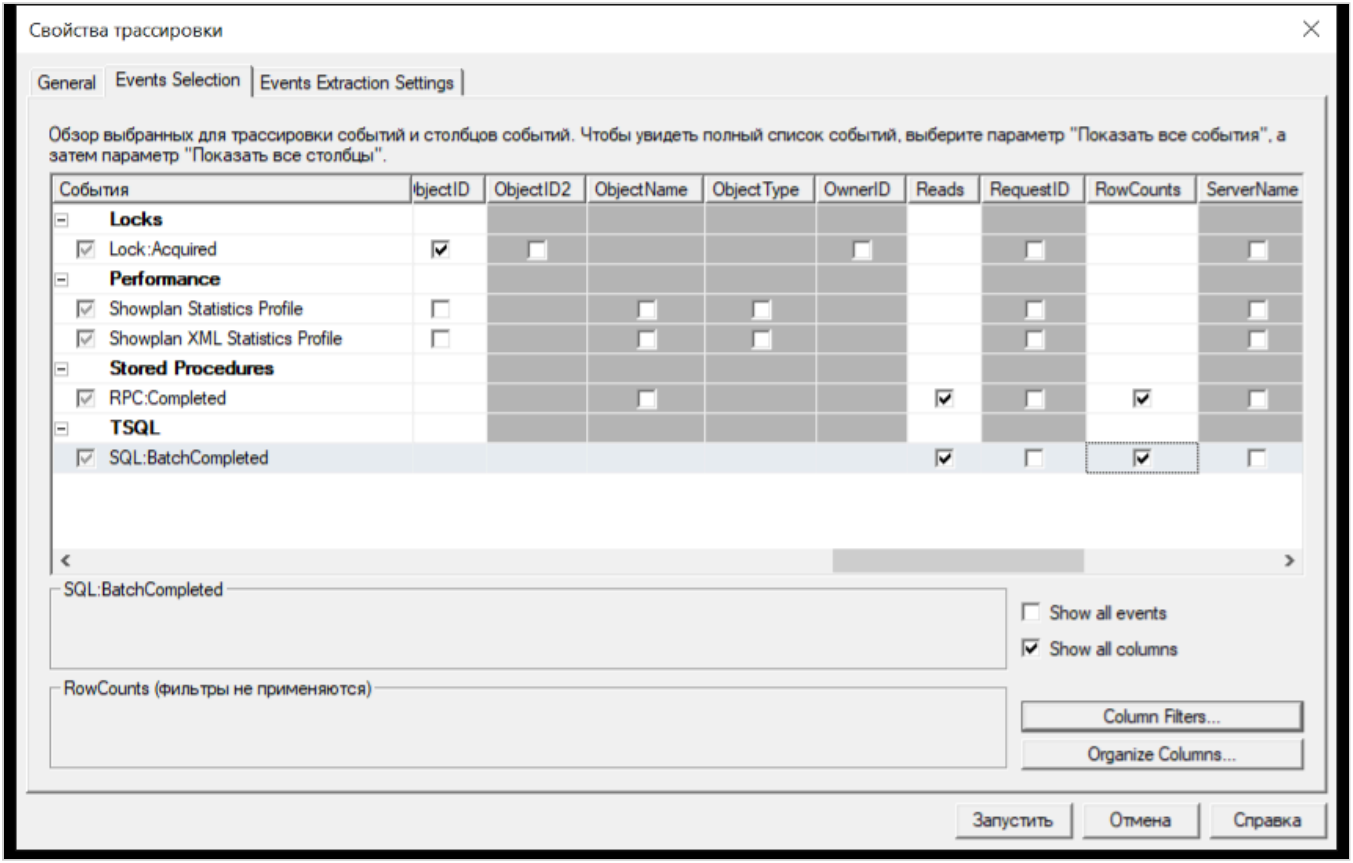
Отличия от задания:

- Событие Lock:Acquired не имеет колонки DatabaseName, поэтому добавлен фильтр DatabaseID, иначе в трассировку попали бы блокировки всех баз.
- У Showplan-событий нет Duration, Reads и CPU, а у Showplan Statistics Profile нет TextData: план лежит в BinaryData.

Скриншот 1. Фильтры трассировки.



Скриншот 2. Выбранные события.



4. Проблемные запросы до оптимизации

Запросы из `03_tests.sql` выполнялись в SSMS по 3 раза.

```
SELECT *
FROM Employees
WHERE Salary > 50000;
```

```
SELECT e.FirstName, e.LastName, d.DepartmentName, p.ProjectName, t.HoursWorked
FROM Employees e
```

```
JOIN Departments d ON e.DepartmentID = d.DepartmentID
JOIN Timesheets t ON e.EmployeeID = t.EmployeeID
JOIN Projects p ON t.ProjectID = p.ProjectID
WHERE t.WorkDate BETWEEN '20230101' AND '20231231';
```

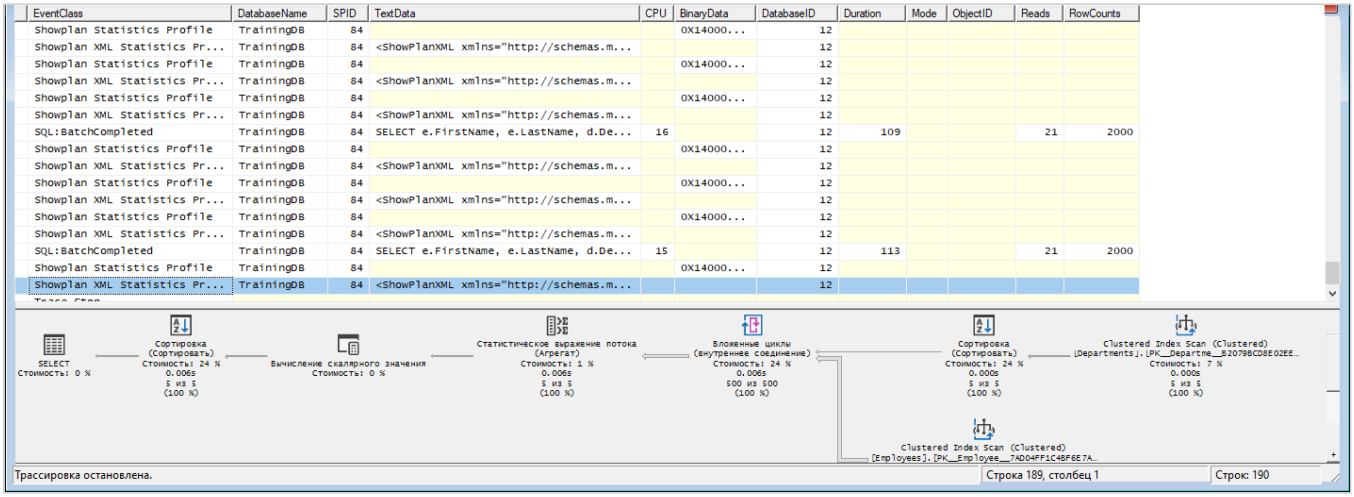
```
SELECT d.DepartmentName, AVG(e.Salary) AS AvgSalary, COUNT(*) AS EmployeeCount
FROM Departments d
JOIN Employees e ON d.DepartmentID = e.DepartmentID
GROUP BY d.DepartmentName
ORDER BY AvgSalary DESC;
```

С порогом 100 мс строки `SQL:BatchCompleted` были только у запроса B: Duration 109 и 113 мс, Reads 21, CPU 15-16, 2000 строк. Запросы A и C порог не прошли, потому что таблицы маленькие.

Для них и для всех следующих сравнений использована серверная трассировка в файл (`05_trace_file.sql`): те же события, фильтр DatabaseID = 12, порог Duration убран. В таблице приведена медиана трёх запусков (`07_compare.sql`). Сортировка по Duration выполнена запросом к файлам трассировки.

Запрос	Duration, мс	CPU, мс	Reads	Строк
B	101	15	21	2000
A	57	0	7	256
C	12	15	37	5

Скриншот 3. Трассировка в Profiler: запрос B и план в нижней панели.



5. Анализ планов и блокировок

Планы получены через `04_plans.sql`.

Запрос	Что в плане	Оценка
A	Clustered Index Scan, стоимость 0.0068	Читается вся таблица, это 7 страниц
B	Три Hash Match и четыре Clustered Index Scan, стоимость 0.1687	Нужны почти все строки, сканирование оправдано
C	Clustered Index Scan, Nested Loops, две Sort, Stream Aggregate, стоимость 0.0469	Наибольшая часть стоимости приходится на сортировку и соединение

- **Table Scan** не встречается: у всех таблиц есть кластерный индекс по первичному ключу, поэтому сканирование называется Clustered Index Scan.

- **Оценка строк:** в планах запросов оценка совпала с фактическим числом строк, например 5 из 5 и 500 из 500 на скриншоте 3, поэтому проблем со статистикой нет.
- **Блокировки:** за три прогона записано 14 093 события Lock:Acquired, большая часть без ObjectID (значение 0 или пусто). Например, ObjectID 1253579504 определён через `OBJECT_NAME` как таблица Employees, 25 событий. На чтении блокировки короткие и задержек не вызывали.
- **Взаимоблокировки:** запросы шли в одной сессии, взаимоблокировок быть не могло, событие Deadlock graph в список не входило.

Главное наблюдение: Duration здесь отражает в основном передачу строк в SSMS. Запрос B с 21 чтением длится дольше запроса C с 37 чтениями, потому что возвращает 2000 строк вместо 5. На таких данных надёжнее сравнивать Reads.

6. Меры оптимизации

Индексы из задания (`02_objects.sql`):

```
CREATE INDEX IX_Employees_Salary ON Employees(Salary);
CREATE INDEX IX_Timesheets_WorkDate ON Timesheets(WorkDate);
```

Они выбраны по полям фильтрации запросов A и B.

Дополнительные покрывающие индексы (`06_covering.sql`). Они добавлены, потому что индексы из задания не изменили планы:

```
CREATE INDEX IX_Employees_Salary_Cover ON Employees(Salary)
INCLUDE (FirstName, LastName, DepartmentID, HireDate);
CREATE INDEX IX_Timesheets_WorkDate_Cover ON Timesheets(WorkDate)
INCLUDE (EmployeeID, ProjectID, HoursWorked);
CREATE INDEX IX_Employees_DepartmentID ON Employees(DepartmentID)
INCLUDE (Salary);
```

7. Результаты после оптимизации

Повторные прогоны сняты серверной трассировкой с теми же событиями. Окно Profiler не удалось перезапустить, поэтому трассировки сохранены в файлы и прочитаны через `fn_trace_gettable`.

Запрос	Без индексов: Duration / Reads	Индексы задания: Duration / Reads	Покрывающие: Duration / Reads
A	57 мс / 7	64 мс / 7	56 мс / 5
B	101 мс / 21	100 мс / 21	89 мс / 20
C	12 мс / 37	9 мс / 37	1 мс / 14

Уменьшилась ли Duration? С индексами задания нет: разница в пределах случайных колебаний. С покрывающими индексами запрос C ускорился с 12 до 1 мс, B с 101 до 89 мс, для A изменения нет.

Стало ли меньше Reads? С индексами задания нет. С покрывающими A 7 → 5, B 21 → 20, C 37 → 14.

Появился ли Index Seek? С индексами задания нет, планы A и B остались прежними: для `SELECT *` по такой маленькой таблице чтение всех 7 страниц дешевле, чем поиск с обращением к строкам, а условие B покрывает все строки табеля. Это подтверждают и узкие варианты в `04_plans.sql` (около 7 сотрудников и 35 строк табеля): план тоже остался со сканированием. С покрывающими индексами Index Seek появился в запросах A и B, в запросе C Index Scan по меньшему индексу заменил сканирование всей таблицы.

Скриншот 4. Сравнение трёх этапов, медиана трёх запусков.

```
        (N'C', N'SELECT d.%')
    ) q(Query, Pattern)
    CROSS APPLY (
        SELECT TOP (1)
            COUNT(*) OVER () AS Runs,
            PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY r.Duration / 1000) OVER () AS MedianDurationMs,
            PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY r.CPU) OVER () AS MedianCPU,
            PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY r.Reads) OVER () AS MedianReads,
            r.RowCounts
        FROM (
            SELECT Duration, CPU, Reads, RowCounts,
                ROW_NUMBER() OVER (ORDER BY StartTime DESC) AS Rn
            FROM fn_trace_gettable(N'C:\Users\Public\tr\' + s.FileName + N'.trc', 1)
            WHERE EventClass = 12
              AND CAST(TextData AS NVARCHAR(4000)) LIKE q.Pattern
        ) r
        WHERE r.Rn <= 3
    ) x
    ORDER BY s.N, q.Query;
    GO
```

Проблемы не найдены. Стр: 31, Симв: 1 Пробелы LF UTF-8 with BOM

Результаты Сообщения

	Test	Stage	Query	Runs	MedianDurationMs	MedianCPU	MedianReads	RowCounts
1	5.1 Сравнение трассировок	1 Без индексов	A	3	57	0	7	256
2	5.1 Сравнение трассировок	1 Без индексов	B	3	101	15	21	2000
3	5.1 Сравнение трассировок	1 Без индексов	C	3	12	15	37	5
4	5.1 Сравнение трассировок	2 Индексы задания	A	3	64	0	7	256
5	5.1 Сравнение трассировок	2 Индексы задания	B	3	100	15	21	2000
6	5.1 Сравнение трассировок	2 Индексы задания	C	3	9	15	37	5
7	5.1 Сравнение трассировок	3 Покрывающие индексы	A	3	56	0	5	256
8	5.1 Сравнение трассировок	3 Покрывающие индексы	B	3	89	16	20	2000
9	5.1 Сравнение трассировок	3 Покрывающие индексы	C	3	1	0	14	5

Запрос успешно выполнен. localhost (17.0 RTM) DESKTOP-02P5L82\nazva ... MyDB 00:00:00 Строка: 1, столбец: 1 9 строки

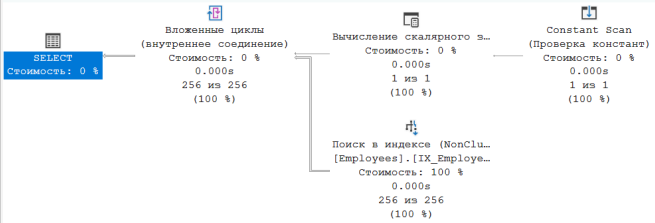
Скриншот 5. Фактический план запроса A с покрывающим индексом: поиск в индексе IX_Employees_Salary_Cover.

```
USE TrainingDB;
GO
SELECT *
FROM Employees
WHERE Salary > 50000;
GO
SELECT e.FirstName, e.LastName, d.DepartmentName, p.ProjectName, t.HoursWorked
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
JOIN Timesheets t ON e.EmployeeID = t.EmployeeID
JOIN Projects p ON t.ProjectID = p.ProjectID
WHERE t.WorkDate BETWEEN '20230601' AND '20230607';
GO
```

Проблемы не найдены. Стр: 1, Симв: 1 Табуляция LF UTF-8 with BOM

Результаты Сообщения План выполнения

```
Запрос 1: стоимость запроса (по отношению к пакету): 11%
SELECT * FROM [Employees] WHERE [Salary]>@1
```




```
Запрос 2: стоимость запроса (по отношению к пакету): 89%
SELECT e.FirstName, e.LastName, d.DepartmentName, p.ProjectName, t.HoursWorked FROM Employees e JOIN Departments d ON e.DepartmentID = d.DepartmentID JOIN
```

Запрос успешно выполнен. localhost (17.0 RTM) DESKTOP-02P5L82\nazva ... TrainingDB 00:00:01 Строка: 1, столбец: 1 291 строки

8. Выводы

- Запрос C ускорился сильнее всего: Reads снизились на 62 %, с 37 до 14, Duration с 12 до 1 мс. Наибольший эффект дал индекс по полю соединения DepartmentID с включённой колонкой Salary.
- Индекс на поле фильтрации помогает не всегда. На 7-10 страницах и при выборке большой доли строк оптимизатор законно выбирает сканирование.

- Покрывающий индекс полезнее простого, когда запрос можно полностью выполнить по индексу. Но каждый индекс замедляет запись и занимает место, а `SELECT *` мешает покрытию.
- Для реальных приложений вывод из шаблона задания верен на больших таблицах: отсутствие индекса на поле фильтрации сильно увеличивает стоимость запроса. Эффект надо проверять на объёмах данных, близких к настоящим, и смотреть на Reads и план, а не только на Duration.
- Порог Duration нужно подбирать под данные: с порогом 100 мс запросы A и C в трассировку не попали.